# NOVAID

## Cahier des Charges

| Informations | |
|---|---|
| Projet | NOVAID |
| Université | TEK-UP University |
| Module | Projet Java |
| Groupe | Ichrak Benali, Yosser Kallel, Zaineb Ben Dhaw, Aymen Dridi |
| Encadrant | M. Mohamed Alouani |
| Date | 14 Avril 2026 |

## SOMMAIRE :

1-Présentation du projet ……..

2-État de l'art — Solutions existantes…….

3-Acteurs du système…….

4-Besoins fonctionnels……..

5-Besoins non fonctionnels……..

6-Diagramme des cas d'utilisation……

7- Diagramme de classes…….

8-Choix technologiques…….

## 1. Présentation du projet

Le projet consiste à développer une solution de gestion et de suivi des interventions terrain sur mesure pour **NOVAID**.

Cette application permettra de gérer de bout en bout l'accompagnement des bénéficiaires / clients, d'enregistrer les visites avec des preuves de présence (géolocalisation), et d'assurer une traçabilité complète des interventions. Le système ira au-delà de la simple gestion de données (CRUD) en automatisant la génération de documents, les notifications, et en s'appuyant sur l'Intelligence Artificielle pour structurer les retours terrains. L'objectif est d'abandonner les processus papier pour numériser, accélérer et fiabiliser la prise de décision.

## 2. État de l'art — Solutions existantes

Une analyse comparative des solutions existantes sur le marché permet de mieux positionner notre future application :

Tableau 1:Positionnement de la NOVAID face aux standards du marché

| Solution | Ce qu'elle fait | Limite |
|---|---|---|
| **KoboToolbox / ODK** | Applications très robustes pour la collecte de données mobile | Aspect purement formulaire (Data-entry), manque de restitution (Dashboard, Impact Stories), customisation Backend coûteuse |
| **Salesforce Field Service** | Leader avec écosystème, suivi GPS et assignations | Coût de licence très élevé, solution lourde pour intégration spécifique rapide |
| **Notre Solution pour NOVAID** | Développement spécifique en **Java**, pérennité, pleine propriété des données, adaptation des flux métiers (export, mail, IA) | Coût d'exploitation réduit, solution modulaire |

## 3. Acteurs du système

L'application implique différents profils d'utilisateurs avec des droits spécifiques :

1. **L'Agent / Bénévole Terrain**:

Accède à ses missions en cours, consulte les fiches de base de ses bénéficiaires, effectue des "check-in GPS", et saisit des rapports/notes de visite enrichis.

2. **Le Coordinateur** :

Gère les bénéficiaires de sa zone, assigne les missions aux agents sur le terrain, analyse la heatmap (carte de température des besoins) et génère des notifications et envois de courriels partenaires.

3. **L'Administrateur** :

Possède les droits totaux. Il gère la configuration générale, lance des exports critiques à l'échelle de **NOVAID** et orchestre l'import en masse des données lors de la transition.

## 4. Besoins fonctionnels

- **Gestion métier (CRUD Base) :**

  Création, gestion, mise à jour, et suppression logique des entités (Utilisateurs, Bénéficiaires, Visites, Stocks).
- **Paradigme Collections Avancées:**

  Exploitation systématique de Java 8 Stream API (filter, map, reduce, collect) pour traiter les listes en mémoire (statistiques de visites, génération des alertes pour visites non effectuées) afin de réduire la complexité algorithmique par rapport aux boucles classiques.

### Fonctionnalités "Business" Avancées :

- **Envoi d'e-mails (Notification) :**

  Alerte instantanée aux coordinateurs lors d'une intervention "Urgente".
- **Import/Export de fichiers :**

  Import massif d'une flotte de bénéficiaires via fichier CSV/Excel, et export de rapports .
- **Intégration IA :**

  Un module connecté à des API de LLM (ex. ChatGPT) pour transformer des notes de terrain abréviées dictées par les agents en "Stories" d'impact formatées et prêtes pour la communication de “NOVAID”.

## 5. Besoins non fonctionnels

- **Conception Orientée Objet (POO) & Design Patterns :**

  Découpage strict garantissant l'Encapsulation , l'Héritage, et le Polymorphisme. De plus, le projet devra s'appuyer sur le “Pattern Singleton” de manière impérative pour encapsuler et gérer l'unique instance de connexion à la base de données.
- **Sécurité & Base de données :**

  Tous les mots de passe doivent être hachés. L'intégrité systémique est gérée par une BDD centralisée invoquée via le Singleton.
- **Performance et Fiabilité :**

Usage du multi-threading côté backend et traitement optimisé de gros volumes (streams).

## 6. Diagramme des cas d'utilisation

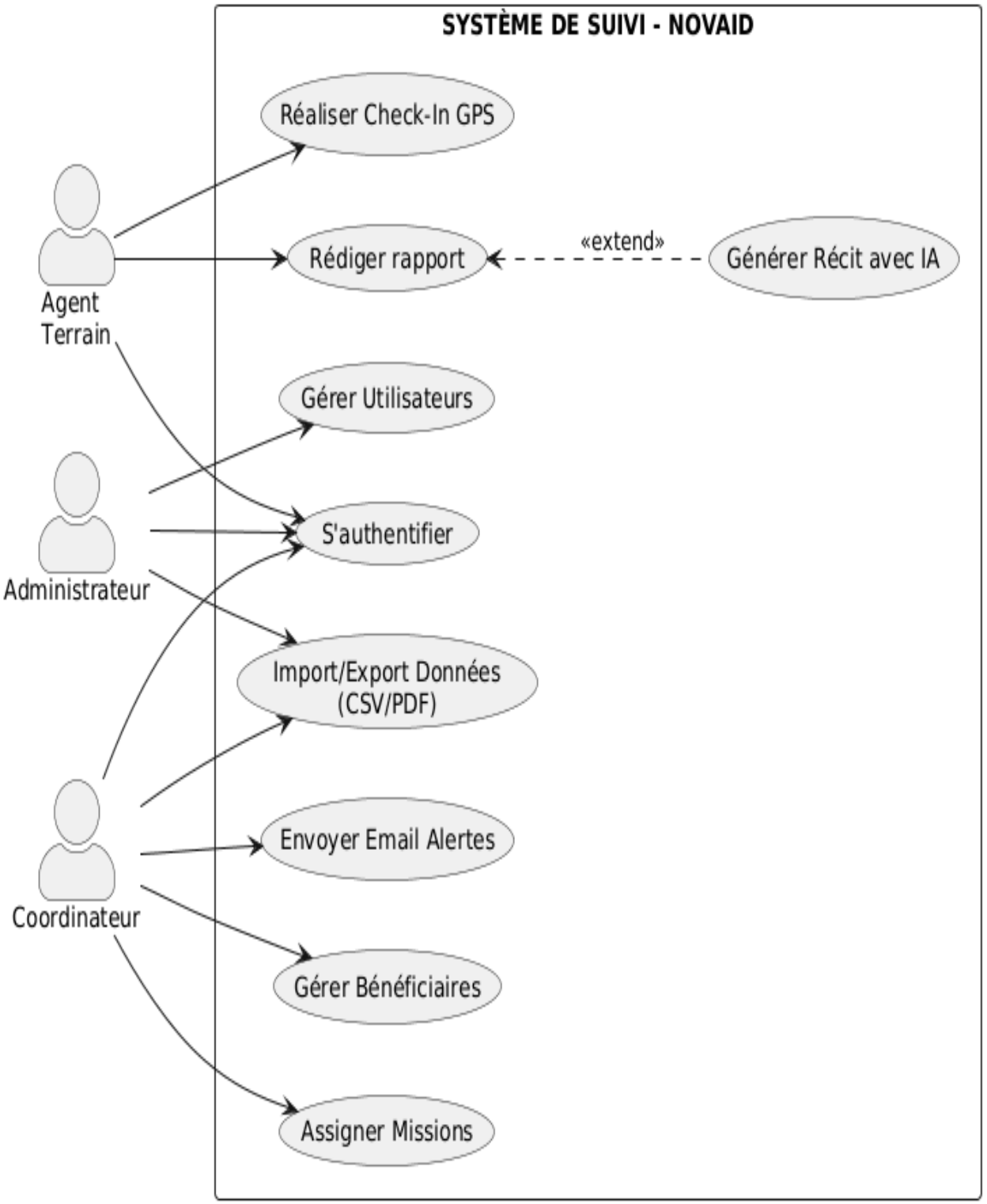


*Figure 1 – Diagramme de cas d'utilisation (Use Case Diagram)*

# 7. Diagramme de classes

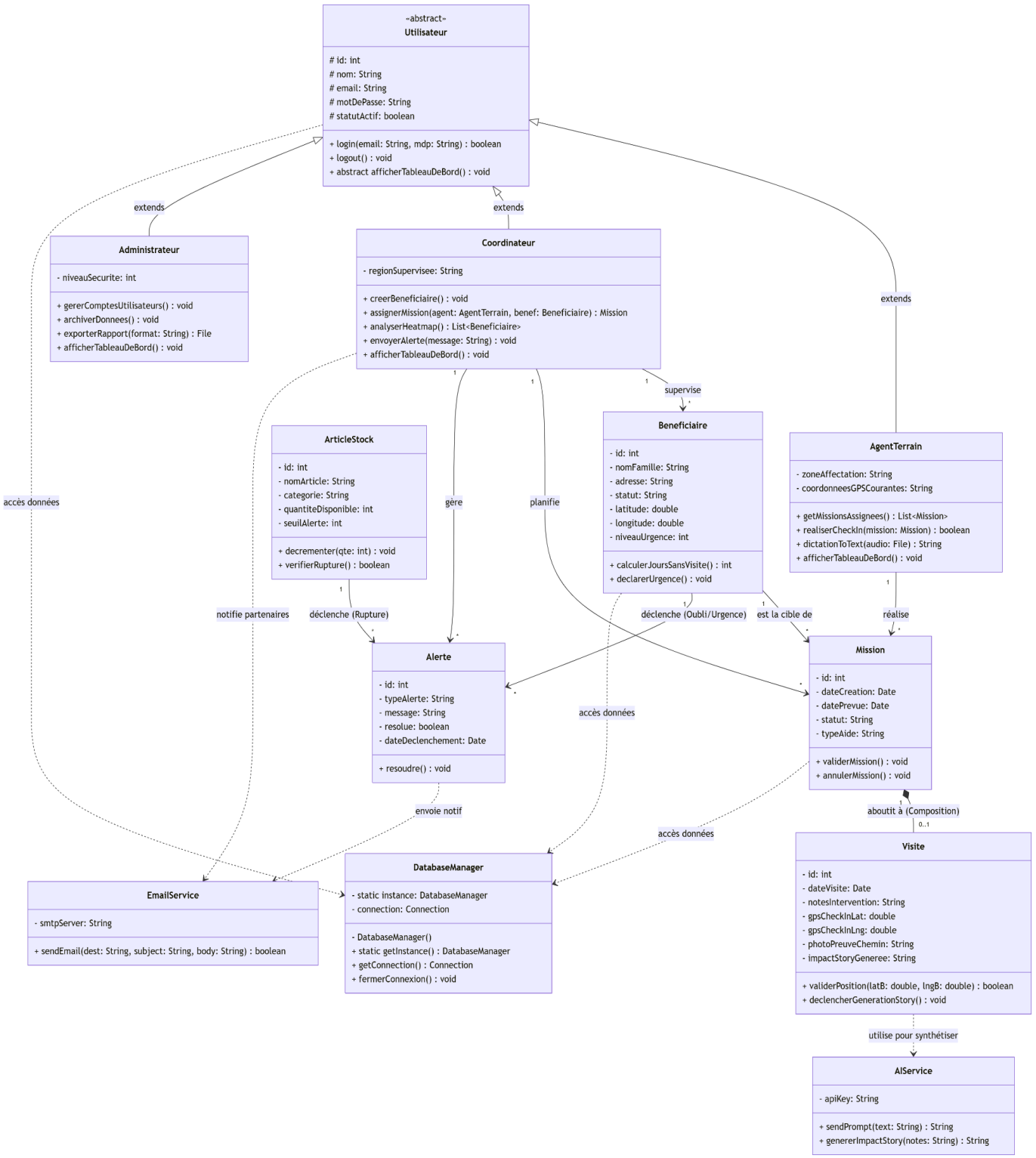


*Figure 2 – Diagramme de classe (Class Diagram)*

# 8. Choix technologiques

## 8.1 Technologies du projet

Tableau 2:Architecture et stack technique de NOVAID

| Composant | Technologies / Outils | Détails |
|---|---|---|
| **Backend** | Java | Utilisation obligatoire des classes/objets, encapsulation, polymorphisme et Stream API |
| **Frontend** | HTML5, CSS3 | Interface web épurée et responsive |
| **Base de données** | MySQL (XAMPP) ou MongoDB | Intégration JDBC ou connecteur natif avec implémentation du pattern **Singleton**, 100% Free et Open Source |
| **Fonctionnalités Avancées** | JavaMail API, Apache POI | Envoi de mails et import/export de fichiers |
| **Cartographie** | Leaflet ou Google Maps API | Affichage et gestion de cartes interactives |

## 8.2 Outils de collaboration

Tableau 3:Environnement de travail collaboratif

| Domaine | Outils | Détails |
|---|---|---|
| **Gestion du code source** | Git, GitHub, GitLab | Utilisation de Git avec une plateforme, flux de travail en branches (Git Flow) |
| **Gestion des Tâches** | Jira, Trello | Organisation en Sprints, suivi des tickets d'anomalies et d'évolution |
| **Communication de l'équipe** | Discord, Microsoft Teams | Communication avec intégration de webhooks GitHub pour notifier les avancées |
| **Conception visuelle** | Figma, Draw.io, Mermaid.js | Maquettes d’interface et diagrammes UML / Base de données |